

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет
имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики
Кафедра технической кибернетики

Отчет по лабораторной работе №2

Дисциплина: «Инженерия данных»

Тема: «Освоение оркестровки в p8n: триггеры, ветвления, бинарные
данные, обработка ошибок»

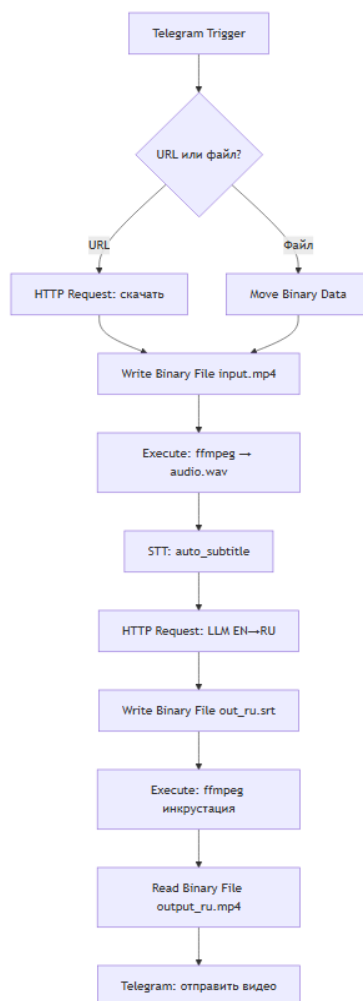
Выполнила: Маргарян Г.А.
Группа: 6232-010402D

Задание на лабораторную работу

1. Telegram Bot принимает либо ссылку на видео, либо сам файл видео.
2. Если пришла ссылка — скачать видео; если пришёл файл — использовать его напрямую. В обоих случаях извлечь аудиодорожку с помощью ffmpeg.
3. Сгенерировать субтитры (EN), используя auto_subtitle.
4. Выполнить перевод EN→RU с помощью LLM из HuggingFace (допустим API; предпочтительно — собственный сервер vLLM/LLama).
5. Добавить полученные (RU) субтитры в исходное видео.
6. Отправить результат обратно пользователю в Telegram.

Допускается работать только с английской речью на входе; многоязычие — как бонус.

Необходимо реализовать следующий пайплайн:



Ход выполнения лабораторной работы

В данной лабораторной работе реализована оркестровка в p8n, которая позволяет отправлять телеграм боту видео/ссылку на видео (я обрабатывала ссылки на YouTube) на иностранном языке и получать от него файл с наложенными субтитрами.

Для выполнения лабораторной работы были использованы следующие инструменты:

1. Docker Desktop – необходим для запуска контейнера
2. p8n «оркестратор» – является «мозгом»: запускает триггер, принимает данные от пользователя, вызывает внешние модели/сервисы, для принятия решений использует ветвления, возвращает ответ пользователя, обрабатывает ошибки.
3. ngrok – сетевой туннель для связи сервера Telegram с локальным сервером. Изначально не использовался, но при отправке сообщения телеграм боту возникала ошибка Telegram Trigger: Bad Request: bad webhook: An HTTPS URL must be provided for webhook, поэтому для решения данной проблемы было принято решение использовать ngrok.
4. yt-dlp – используется для скачивания видео из ссылки на YouTube.
5. FFmpeg – используется для получения аудиодорожки видео и наложения субтитров на видео.
6. Whisper – принимает файл с аудиодорожкой и извлекает из него слова с таймкодами.
7. Llama – необходима для перевода иностранного текста на русский язык: в данной работе изначально предполагалось использовать версию 3.1, но так как все запускалось на CPU, модель оказалась тяжелой, поэтому было принято решение использовать модель поменьше (llama3.2:3b).

Результаты:

Полная схема workflow выглядит следующим образом:

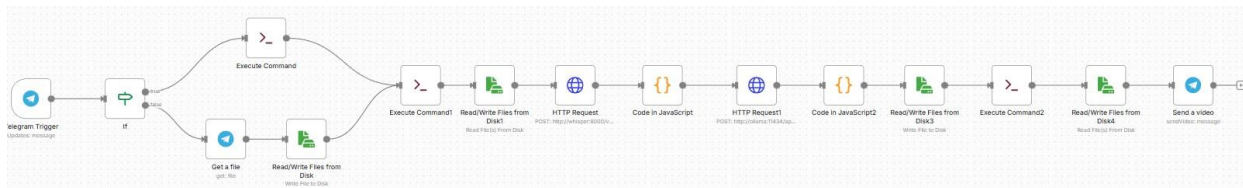


Рисунок 1 – Схема workflow

Схема начинается с добавления триггера на сообщение пользователя в Telegram. Для того, чтобы триггер сработал, необходимо было указать данные телеграм бота (токен бота и id чата). Также на этом этапе использовался сетевой туннель ngrok.

Затем использовалась нода IF, от которой идут две ветки: в случае, когда пользователь отправил телеграм боту видеофайл – движемся по ветке false; в случае, когда телеграм боту пользователь отправил ссылку на видео (например, ссылка на YouTube) – движемся по ветке true.

Рассмотрим ветку false: когда пользователь отправляет телеграм боту видеофайл срабатывает нода для Telegram – Get a file, которая сохраняет видео сразу, а потом с помощью ноды Write file to disk сохраняется в указанную директорию.

Теперь рассмотрим ветку true: когда пользователь отправляет телеграм боту ссылку на видео, а в данной работе я обрабатывала ссылки из YouTube, то в ноде Execute Command с помощью команды ffmpeg видео скачивается и также сохраняется в указанную директорию.

Из сохраненного в директории видеофайла с помощью ноды Execute Command1 извлекается аудиодорожка. Нода Write file to disk сохраняет эту аудиодорожку как файл с расширением .wav.

Далее добавлена нода HTTP Request, которая обращается к Whisper (использовалась версия tiny). Модель предназначена для распознавания речи. Для того, чтобы из аудиодорожки была правильно извлечена речь с таймкодами и сохранена в файл с расширением .srt, использовалась нода Code

in JavaScript. Можно было вместо тандема нод HTTP Request и Code in JavaScript использовать только ноду Execute Command, но у меня не получалось извлечь из аудио слова, поэтому было принято решение отказаться от этой идеи.

После того, как был получен файл со словами, он передавался в ноду HTTP Request1, которая обращается к модели Ollama. Модель используется для перевода полученного текста. Изначально предполагалось использовать версию модели Llama3.1, но так как лабораторная работа выполнялась на CPU было принято решение использовать модель поменьше, а именно Llama3.2:3b. Здесь также использовалась нода Code in JavaScript2 для того, чтобы текст был правильно переведен и сохранен в директории.

С помощью ноды Execute Command2 переведенный текст накладывается на видео. Нода Read files from disk извлекает итоговое видео с наложенным текстом. В результате телеграм бот отправляет пользователю скачанный видеофайл с наложенными на него субтитрами на русском языке.

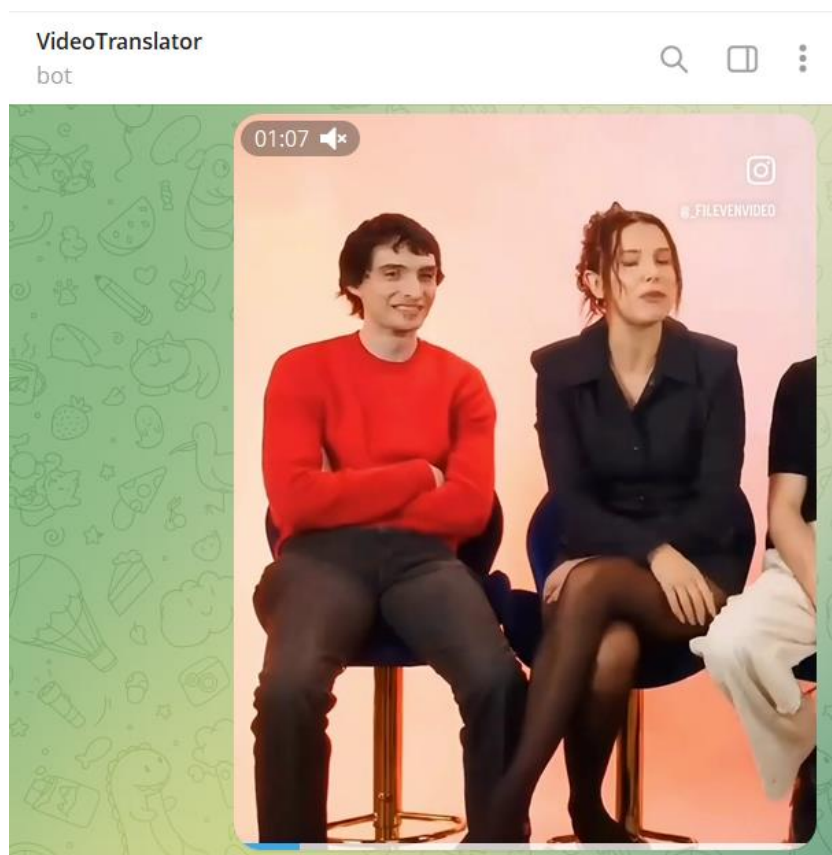


Рисунок 2 – Пользователь отправил телеграм боту видеофайл

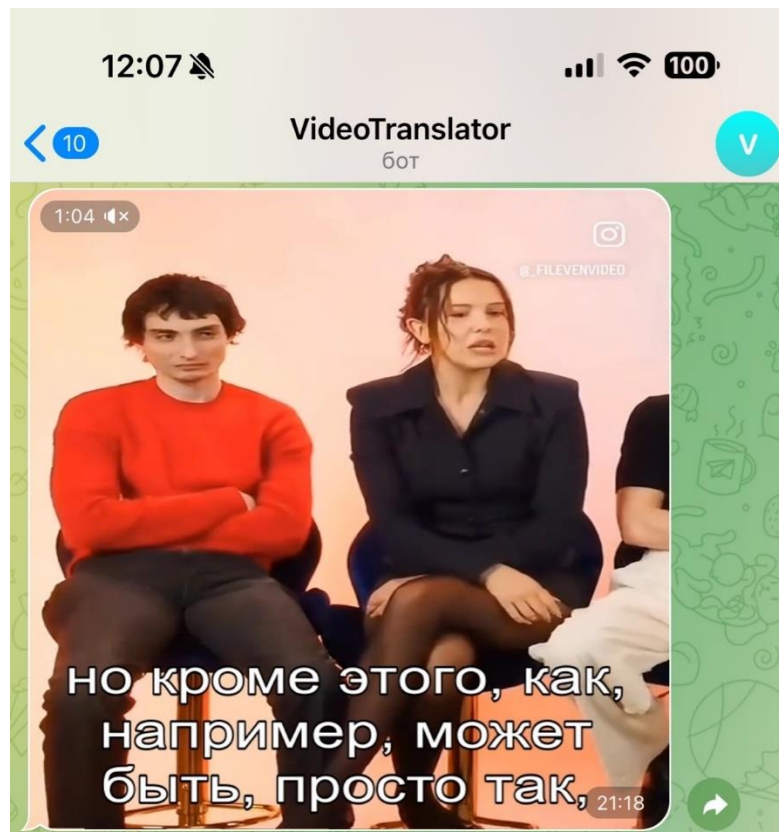


Рисунок 3 – Телеграм бот получил от пользователя видеофайл на английском языке и отправил пользователю в ответ файл с субтитрами

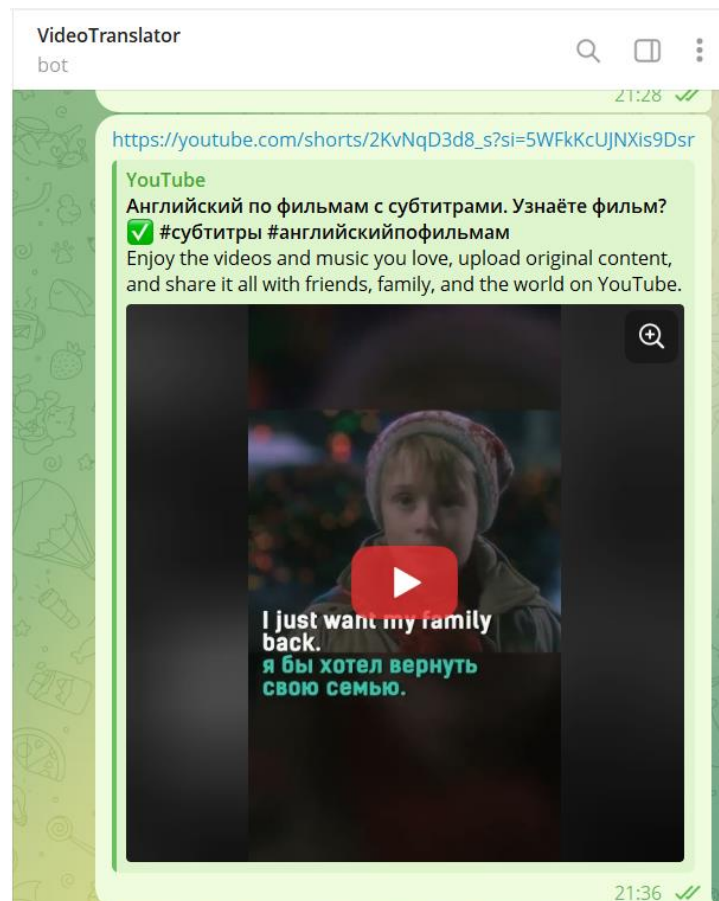


Рисунок 4 – Пользователь отправил телеграм боту ссылку на видео

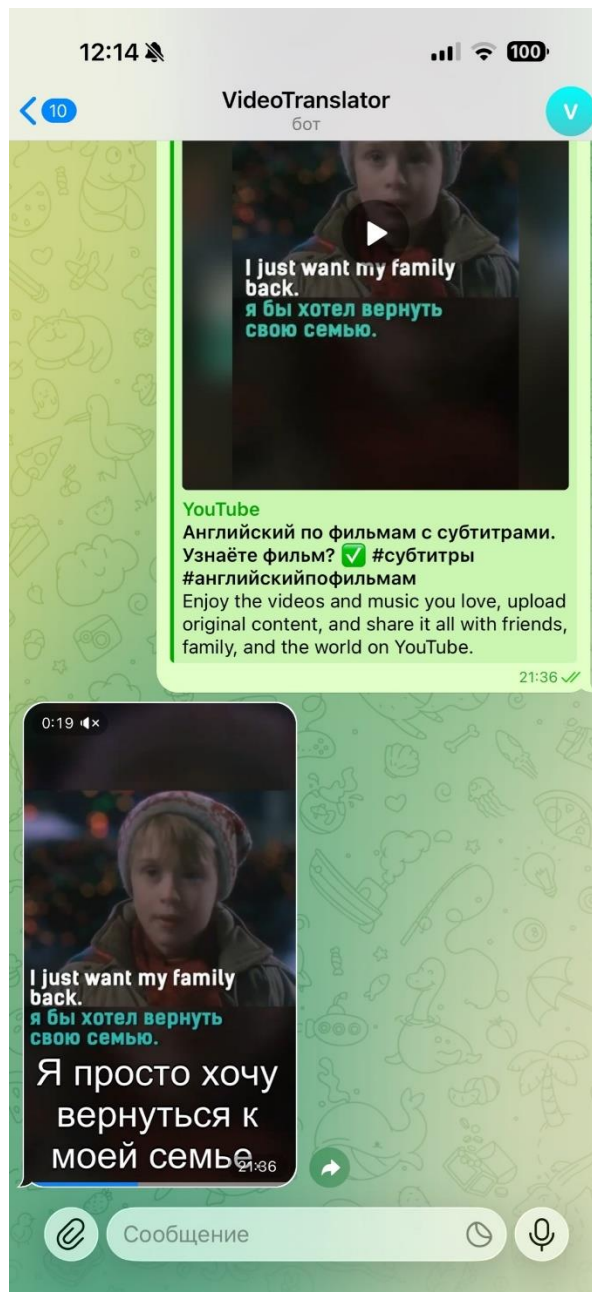


Рисунок 5 - Телеграм бот получил от пользователя ссылку на видео в youtube на английском языке и отправил пользователю в ответ файл с субтитрами

Выводы

Таким образом, в данной лабораторной работе была полностью освоена оркестровка в n8n. При изучении материалов для выполнения работы использовались как Интернет, так и AI модель DeepSeek.

При выполнении лабораторной работы возникли некоторые *сложности*:

1. В самом начале столкнулась с проблемой, что триггер не реагирует на сообщение пользователя. Пришлось решать данную проблему с помощью сетевого туннеля ngrok, с которым проблемы не закончились, так как периодически он переставал работать (но в целом это было не так критично, потому что это было связано с плохим интернет-соединением).
2. Когда настраивала отправку пользователем ссылки на видео (а я отправляла именно ссылку на youtube), то столкнулась с такой проблемой, что бот не может скачать видео. Поэтому пришлось настраивать yt-dlp.
3. Затем была проблема, что после извлечения звука из видео не получалось из звука извлечь слова и сохранить их в файл для дальнейшей обработки (возможно какие-то проблемы с настройкой были). Пришлось явно указывать эти действия в дополнительной ноде Code in JavaScript.
4. Еще была проблема с тем, что работа выполнялась на CPU и его возможности, к сожалению, не безграничны. Для перевода текста не было возможным использовать большие модели, только маленькие. В результате мы пожертвовали качеством перевода (модель не смогла все перевести).

Работа с n8n мне очень понравилась. Сначала было очень тяжело понять, как это все работает и как правильно настроить все ноды, чтобы получился нужный результат. Но в процессе работы уже пришло понимание и все стало двигаться быстрее.